FactoryPulse

Backend Architecture Overview — Machine Management & Production Orders (v0.3)

Clean Architecture | .NET 10 | Rich Domain Model | EF Core + SQL (Docker)

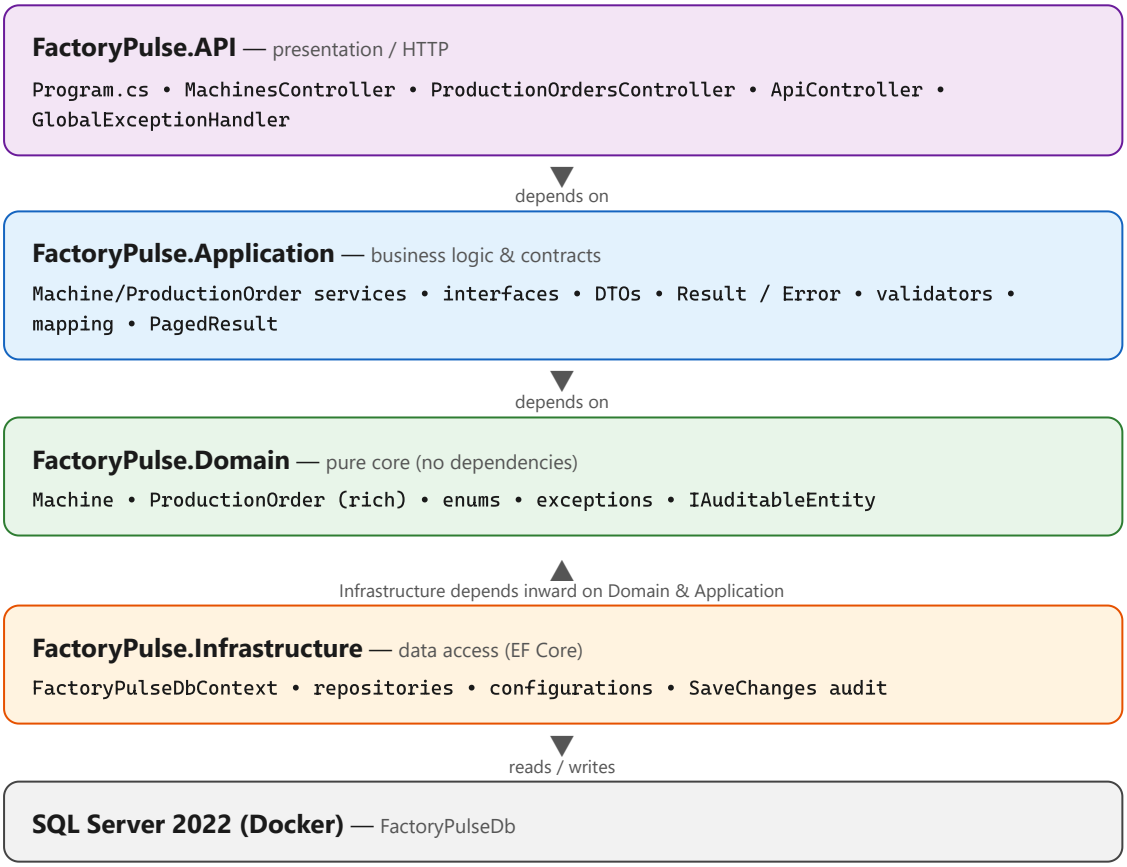
Modular monolith • dependencies point inward • Controller → Service → Repository → EF Core → SQL Server

---

1. The big idea

FactoryPulse uses **Clean Architecture**: a layered design where **all dependencies point inward** toward a pure, framework-free domain. The solution is split into **four projects**, one per layer, so the boundaries are enforced at compile time — the Domain project literally cannot reference EF Core.

API (HTTP) Application (business logic) Infrastructure (data access) Domain (pure core)



> **Reference rule:** API → Application + Infrastructure • Application → Domain • Infrastructure → Domain + Application • **Domain → nothing**.

2. What lives in each layer

Domain — what the business *is*

ELEMENT	KIND	PURPOSE
`Machine`	Entity (anemic)	Factory machine. `Id`, `Name`, `Description?`, `Status`, audit fields.
`ProductionOrder`	Entity (rich)	Order with a lifecycle. Private setters; created via `Create(...)`; state changed only via `Start/Complete/Cancel`. FK to `Machine`.
`MachineStatus` / `ProductionOrderStatus`	Enums	Fixed state sets, stored as strings.
`InvalidProductionOrderTransitionException`	Exception	Thrown by the entity if an illegal transition is attempted (invariant backstop).
`IAuditableEntity`	Interface	Marks entities with `CreatedAt` / `UpdatedAt` so the DbContext can set them automatically.

Application — business logic & contracts

ELEMENT	KIND	PURPOSE
`IMachineService` / `IProductionOrderService`	Interfaces	Application-facing contracts. All operations return `Result<…>`.
`MachineService` / `ProductionOrderService`	Services	Business logic. Depend on repositories + validators. Map entities↔DTOs; enforce cross-aggregate & uniqueness rules; drive the rich model's transitions.
`IMachineRepository` / `IProductionOrderRepository`	Interfaces	Data-access contracts owned by the business layer (ports).
DTOs	Records/POCOs	Separate request/response shapes (`MachineDto`, `CreateProductionOrderRequest`, …). Status exposed as strings.
Validators	FluentValidation	Input-shape rules (required, lengths, quantity > 0).
`Result` / `Result<T>`, `Error`, `ErrorType`, `Errors`	Common	Outcome container with categories & a central error catalog.
`PagedResult<T>`, `ProductionOrderQueryParameters`	Common	Pagination (self-clamping page size) and filtering.

Infrastructure — *how* data is stored

ELEMENT	KIND	PURPOSE
`FactoryPulseDbContext`	DbContext	`DbSet`s for Machine & ProductionOrder. Overrides `SaveChangesAsync` to set audit fields for any `IAuditableEntity`.
`MachineConfiguration` / `ProductionOrderConfiguration`	EF config	Fluent mapping: lengths, string enums, the FK relationship, the unique index on `OrderNumber`.
`MachineRepository` / `ProductionOrderRepository`	Repositories	EF Core queries. Paging/filtering, eager-loading (`Include`), uniqueness checks; stage-then-save.

API — *how* it is exposed

ELEMENT	KIND	PURPOSE
`ApiController`	Base controller	`HandleFailure` maps `ErrorType` → HTTP status; returns all validation errors as `ValidationProblemDetails`.
`MachinesController` / `ProductionOrdersController`	Controllers	Thin: call service → `Result.Match(...)` → HTTP. Orders add transition sub-resources and pagination/filtering.
`GlobalExceptionHandler`	Middleware	Catches unexpected exceptions → logs → 500 ProblemDetails.

3. Outcomes: the Result pattern

Every service method returns a **Result** — "succeeded (here's the value)" or "failed (here are the errors)". Unexpected failures still throw and are handled by middleware. `Error` carries a category so the controller picks the right status:

ERRORTYPE	HTTP
`NotFound`	404
`Validation`	400 (all errors listed)
`Conflict`	409
`Failure`	500

4. Production Orders — relationship & lifecycle

A `Machine` has many `ProductionOrder`s; each order belongs to one machine (FK `MachineId`, `OnDelete: Restrict` — a machine with orders cannot be deleted).

The state machine (rich domain model)

The order's `Status` can only change through behaviour methods, each guarded by a `Can*` rule. Illegal transitions are impossible from the API and throw inside the entity as a backstop.

`Planned` —Start()→ `Running` —Complete()→ `Completed`

Planned / Running —Cancel()→ `Cancelled` (terminal — cannot be restarted)

Business rules — where each lives

RULE	LAYER	RESULT
Quantity > 0	Validator	400
Completed order needs a valid end date	Service + Domain	400
Order number unique	Service + DB unique index	409
No order on a retired machine	Service (loads the Machine)	400 / 409
Cancelled order can't be restarted	Domain (`CanStart`)	409

> **Placement principle:** input-shape rules → **validator**; rules needing other data → **service**; rules about an entity's own lifecycle → the **entity**.

5. Request flow — `POST /api/orders`

1. **API · PRODUCTIONORDERSCONTROLLER.CREATE**
   Binds the body to `CreateProductionOrderRequest`, calls the service.
2. **APPLICATION · VALIDATE & CHECK RULES**
   Validator (quantity, lengths). Then service loads the `Machine` (must exist, not retired) and checks order-number uniqueness.
3. **APPLICATION · BUILD VIA FACTORY**
   `ProductionOrder.Create( ... )` → a valid order in `Planned`.
4. **INFRASTRUCTURE · REPOSITORY + DBCONTEXT**
   `AddAsync` stages; `SaveChangesAsync` commits — and sets audit fields for the new order.
5. **EF CORE → SQL SERVER**
   Sequential GUID key generated; `INSERT` in one transaction; unique index enforces order-number uniqueness.
6. **API · TRANSLATE TO HTTP**
   `result.Match( ... )` → **201 Created** with a `Location` header, or 400/409 for a rule failure.

6. Dependency injection (per request, Scoped)

NEEDS	GETS	REGISTERED IN
`IProductionOrderService`	`ProductionOrderService`	`AddApplication()`
`IProductionOrderRepository`	`ProductionOrderRepository`	`AddInfrastructure()`
validators	auto-registered from the assembly	`AddApplication()`
`FactoryPulseDbContext`	SQL Server context	`AddInfrastructure()`

> A controller asks for a service; DI builds it with its repositories, validators, and the `DbContext` — the whole chain, all sharing one scope (one unit of work) per request.

7. Quality & cross-cutting

- **Testing:** xUnit + NSubstitute + Shouldly. Unit tests cover the `ProductionOrder` state machine, the `Result` pattern, and service business rules (repositories mocked).
- **Logging:** Serilog to console + rolling file. Controllers silent; services log business events; middleware logs exceptions.
- **Audit fields:** `CreatedAt` / `UpdatedAt` set centrally in `SaveChanges` via `IAuditableEntity`.
- **Persistence:** SQL Server 2022 in Docker; EF Core migrations; `Status` stored as text; secrets via `.env` + user-secrets.
